

PHP basics

1. Introduction to PHP

PHP is a **server-side scripting language** used to build dynamic websites.

PHP can:

- Generate HTML dynamically
- Handle forms
- Work with user input
- Manage sessions and cookies
- Connect to databases later
- Build login systems, dashboards, blogs, and web apps

A PHP file ends with:

```
.php
```

Basic PHP code:

```
<?php  
echo "Hello, PHP!";  
?>
```

2. PHP Setup

To run PHP locally, you can use:

- XAMPP

A typical project folder:

```
php-workbook/  
|  
└─ index.php
```

Example file:

```
<?php  
echo "PHP is working!";
```

If you use XAMPP, place your project inside:

```
htdocs/
```

Then open in browser:

```
http://localhost/php-workbook/index.php
```

3. PHP Syntax

PHP code starts with:

```
<?php
```

Example:

```
<?php  
echo "Welcome to PHP";
```

Each statement usually ends with a semicolon:

```
echo "Hello";  
echo "World";
```

Comments

Single-line comment:

```
<?php
// This is a comment
echo "Learning PHP";
```

Multi-line comment:

```
<?php
/*
This is a multi-line comment.
It can explain a larger block of code.
*/
echo "PHP comments are useful.";
```

PHP with HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>PHP Page</title>
</head>
<body>

<h1>Welcome</h1>

<p>
    <?php
    echo "This text came from PHP.";
    ?>
</p>
```

```
</body>
</html>
```

PHP runs on the server, then sends the final HTML to the browser.

4. Variables

Variables store data.

In PHP, variables start with `$`.

```
<?php
$name = "Alex";
$age = 22;

echo $name;
echo $age;
```

Joining text and variables

Use the dot `.` for concatenation:

```
<?php
$name = "Alex";
$age = 22;

echo "Name: " . $name . "<br>";
echo "Age: " . $age;
```

Output:

```
Name: Alex
Age: 22
```

Variables inside double quotes

```
<?php
$name = "Maria";

echo "Hello, $name";
```

Output:

```
Hello, Maria
```

Single quotes vs double quotes

```
<?php
$name = "Maria";

echo "Hello $name <br>";
echo 'Hello $name';
```

Output:

```
Hello Maria
Hello $name
```

Double quotes process variables.

Single quotes print text more literally.

5. Data Types

PHP has several common data types.

Type	Example	Meaning
String	"Hello"	Text

Type	Example	Meaning
Integer	25	Whole number
Float	10.5	Decimal number
Boolean	true / false	Yes or no
Array	["Apple", "Mango"]	Multiple values
NULL	null	No value

Example

```
<?php
$name = "Rahim";
$age = 22;
$cgpa = 3.75;
$isStudent = true;
$middleName = null;

var_dump($name);
echo "<br>";

var_dump($age);
echo "<br>";

var_dump($cgpa);
echo "<br>";

var_dump($isStudent);
echo "<br>";

var_dump($middleName);
```

`var_dump()` shows both the value and data type.

Boolean example

```
<?php
$isLoggedIn = true;

if ($isLoggedIn) {
    echo "Welcome back!";
} else {
    echo "Please login.";
}
```

6. Operators

Operators are used to calculate, compare, and make decisions.

6.1 Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder
**	Power

Example:

```
<?php
$a = 20;
$b = 6;

echo "Addition: " . ($a + $b) . "<br>";
echo "Subtraction: " . ($a - $b) . "<br>";
echo "Multiplication: " . ($a * $b) . "<br>";
echo "Division: " . ($a / $b) . "<br>";
```

```
echo "Remainder: " . ($a % $b) . "<br>";  
echo "Power: " . ($a ** 2);
```

6.2 Assignment Operators

```
<?php  
$score = 10;  
  
$score += 5;  
echo $score . "<br>"; // 15  
  
$score -= 3;  
echo $score . "<br>"; // 12  
  
$score *= 2;  
echo $score . "<br>"; // 24  
  
$score /= 4;  
echo $score . "<br>"; // 6
```

6.3 Comparison Operators

Operator	Meaning
<code>==</code>	Equal value
<code>===</code>	Equal value and same type
<code>!=</code>	Not equal
<code>!==</code>	Not equal value or not same type
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal
<code><=</code>	Less than or equal

Example:

```
<?php
$a = 10;
$b = "10";

var_dump($a == $b);
echo "<br>";

var_dump($a === $b);
```

Output:

```
bool(true)
bool(false)
```

`==` checks only value.

`===` checks value and type.

6.4 Logical Operators

Operator	Meaning
<code>&&</code>	AND
<code>,</code>	
<code>!</code>	NOT

Example:

```
<?php
$username = "admin";
$password = "12345";

if ($username === "admin" && $password === "12345") {
    echo "Login successful";
} else {
```

```
    echo "Invalid login";  
}
```

7. Conditional Statements

Conditional statements help your program make decisions.

7.1 **if**

```
<?php  
$age = 18;  
  
if ($age >= 18) {  
    echo "You are an adult.";  
}
```

7.2 **if...else**

```
<?php  
$age = 16;  
  
if ($age >= 18) {  
    echo "You can vote.";  
} else {  
    echo "You cannot vote yet.";  
}
```

7.3 **if...elseif...else**

```
<?php  
$marks = 82;
```

```

if ($marks >= 80) {
    echo "Grade: A+";
} elseif ($marks >= 70) {
    echo "Grade: A";
} elseif ($marks >= 60) {
    echo "Grade: B";
} elseif ($marks >= 50) {
    echo "Grade: C";
} elseif ($marks >= 40) {
    echo "Grade: D";
} else {
    echo "Grade: F";
}

```

7.4 Nested condition

```

<?php
$username = "admin";
$password = "12345";

if ($username === "admin") {
    if ($password === "12345") {
        echo "Welcome admin.";
    } else {
        echo "Wrong password.";
    }
} else {
    echo "Unknown user.";
}

```

7.5 switch

Use `switch` when you compare one value with many possible cases.

```
<?php
$day = "Friday";

switch ($day) {
    case "Friday":
        echo "Weekend starts!";
        break;

    case "Saturday":
        echo "Rest day.";
        break;

    case "Sunday":
        echo "Work starts.";
        break;

    default:
        echo "Regular day.";
}
```

8. Loops

Loops repeat code.

8.1 **for** loop

Use when you know how many times to repeat.

```
<?php
for ($i = 1; $i <= 5; $i++) {
    echo "Number: $i <br>";
}
```

Output:

```
Number: 1  
Number: 2  
Number: 3  
Number: 4  
Number: 5
```

8.2 **while** loop

Use when repetition depends on a condition.

```
<?php  
$count = 1;  
  
while ($count <= 5) {  
    echo "Count: $count <br>";  
    $count++;  
}
```

8.3 **do...while** loop

Runs at least once.

```
<?php  
$count = 10;  
  
do {  
    echo "Count is $count <br>";  
    $count++;  
} while ($count <= 5);
```

Even though the condition is false, this still runs once.

8.4 **foreach** loop

Best for arrays.

```
<?php
$fruits = ["Apple", "Banana", "Mango"];

foreach ($fruits as $fruit) {
    echo $fruit . "<br>";
}
```

Robust Example: Multiplication Table

```
<?php
$number = 5;

for ($i = 1; $i <= 10; $i++) {
    echo "$number x $i = " . ($number * $i) . "<br>";
}
```

9. Arrays

Arrays store multiple values in one variable.

9.1 Indexed Array

```
<?php
$students = ["Ali", "Nora", "David"];

echo $students[0] . "<br>";
echo $students[1] . "<br>";
echo $students[2] . "<br>";
```

Indexes start from 0.

9.2 Loop through Indexed Array

```
<?php
$students = ["Ali", "Nora", "David"];

foreach ($students as $student) {
    echo "Student: $student <br>";
}
```

9.3 Associative Array

Associative arrays use named keys.

```
<?php
$student = [
    "name" => "Ali",
    "age" => 21,
    "department" => "CSE"
];

echo "Name: " . $student["name"] . "<br>";
echo "Age: " . $student["age"] . "<br>";
echo "Department: " . $student["department"];
```

9.4 Loop through Associative Array

```
<?php
$student = [
    "name" => "Ali",
    "age" => 21,
    "department" => "CSE"
];
```

```
foreach ($student as $key => $value) {  
    echo ucfirst($key) . ": $value <br>";  
}
```

Output idea:

```
Name: Ali  
Age: 21  
Department: CSE
```

9.5 Multidimensional Array

A multidimensional array is an array inside another array.

```
<?php  
$students = [  
    [  
        "name" => "Ali",  
        "marks" => 85  
    ],  
    [  
        "name" => "Nora",  
        "marks" => 92  
    ],  
    [  
        "name" => "David",  
        "marks" => 76  
    ]  
];  
  
foreach ($students as $student) {  
    echo $student["name"] . " scored " . $student["marks"] .  
    "<br>";  
}
```


Robust Example: Student Result Table

```
<?php
$students = [
    ["name" => "Ali", "marks" => 85],
    ["name" => "Nora", "marks" => 92],
    ["name" => "David", "marks" => 38],
    ["name" => "Sara", "marks" => 67]
];
?>

<!DOCTYPE html>
<html>
<head>
    <title>Student Results</title>
</head>
<body>

<h2>Student Results</h2>

<table border="1" cellpadding="10">
    <tr>
        <th>Name</th>
        <th>Marks</th>
        <th>Status</th>
    </tr>

    <?php foreach ($students as $student): ?>
        <tr>
            <td><?php echo $student["name"]; ?></td>
            <td><?php echo $student["marks"]; ?></td>
            <td>
                <?php
                if ($student["marks"] >= 40) {
                    echo "Pass";
                }
            </td>
        </tr>
    </?php>
</table>
```

```

        } else {
            echo "Fail";
        }
    ?>
</td>
</tr>
<?php endforeach; ?>

</table>

</body>
</html>

```

10. Functions

Functions are reusable blocks of code.

10.1 Basic Function

```

<?php
function sayHello() {
    echo "Hello from a function!";
}

sayHello();

```

10.2 Function with Parameter

```

<?php
function greetUser($name) {
    echo "Hello, $name!";
}

```

```
greetUser("Maria");
```

10.3 Function with Return Value

```
<?php
function addNumbers($a, $b) {
    return $a + $b;
}

$result = addNumbers(10, 20);

echo "Result: $result";
```

10.4 Function with Type Hints

```
<?php
function calculateTotal(float $price, int $quantity): float {
    return $price * $quantity;
}

echo calculateTotal(250.50, 3);
```

This makes the function clearer and safer.

10.5 Default Parameter Value

```
<?php
function greet($name = "Guest") {
    echo "Hello, $name!";
}
```

```
greet();  
echo "<br>";  
greet("Amina");
```

Output:

```
Hello, Guest!  
Hello, Amina!
```

Robust Example: Grade Calculator Functions

```
<?php  
function getGrade(int $marks): string {  
    if ($marks >= 80) {  
        return "A+";  
    } elseif ($marks >= 70) {  
        return "A";  
    } elseif ($marks >= 60) {  
        return "B";  
    } elseif ($marks >= 50) {  
        return "C";  
    } elseif ($marks >= 40) {  
        return "D";  
    } else {  
        return "F";  
    }  
}  
  
function getStatus(int $marks): string {  
    return $marks >= 40 ? "Pass" : "Fail";  
}  
  
$students = [  
    ["name" => "Ali", "marks" => 85],
```

```

        ["name" => "Nora", "marks" => 72],
        ["name" => "David", "marks" => 35]
    ];

    foreach ($students as $student) {
        echo $student["name"] . " - ";
        echo "Grade: " . getGrade($student["marks"]) . " - ";
        echo "Status: " . getStatus($student["marks"]) . "<br>";
    }

```

11. Forms and User Input

Forms send user input to PHP.

Common form methods:

Method	Meaning
GET	Sends data through the URL
POST	Sends data secretly through the request body

11.1 GET Form

File 1: **get-form.php**

```

<!DOCTYPE html>
<html>
<head>
    <title>GET Form</title>
</head>
<body>

<h2>Search Form</h2>

```

```
<form action="get-result.php" method="get">
  <label>Search:</label>
  <input type="text" name="keyword">

  <button type="submit">Search</button>
</form>

</body>
</html>
```

File 2: `get-result.php`

```
<?php
$keyword = $_GET["keyword"] ?? "";
$safeKeyword = htmlspecialchars($keyword);
?>

<!DOCTYPE html>
<html>
<head>
  <title>GET Result</title>
</head>
<body>

<h2>Search Result</h2>

<p>You searched for: <?php echo $safeKeyword; ?></p>

</body>
</html>
```

When you submit the form, the URL may look like this:

```
get-result.php?keyword=php
```

11.2 POST Form

File 1: **post-form.php**

```
<!DOCTYPE html>
<html>
<head>
    <title>POST Form</title>
</head>
<body>

<h2>Registration Form</h2>

<form action="post-result.php" method="post">
    <label>Name:</label>
    <input type="text" name="name">
    <br><br>

    <label>Email:</label>
    <input type="email" name="email">
    <br><br>

    <label>Age:</label>
    <input type="number" name="age">
    <br><br>

    <button type="submit">Register</button>
</form>
```

```
</body>
</html>
```

File 2: `post-result.php`

```
<?php
$name = $_POST["name"] ?? "";
$email = $_POST["email"] ?? "";
$age = $_POST["age"] ?? "";

$safeName = htmlspecialchars($name);
$safeEmail = htmlspecialchars($email);
$safeAge = htmlspecialchars($age);
?>

<!DOCTYPE html>
<html>
<head>
    <title>POST Result</title>
</head>
<body>

<h2>Registration Result</h2>

<p>Name: <?php echo $safeName; ?></p>
<p>Email: <?php echo $safeEmail; ?></p>
<p>Age: <?php echo $safeAge; ?></p>

</body>
</html>
```

11.3 Form Validation

Never trust user input directly.

Use:

```
trim()
```

to remove extra spaces.

Use:

```
htmlspecialchars()
```

to safely display user input.

Use:

```
filter_var()
```

to validate email and other values.

Robust Example: Registration Form with Validation

Create one file:

```
register.php
```

```
<?php
$name = "";
$email = "";
$age = "";

$nameError = "";
$emailError = "";
$ageError = "";
$successMessage = "";
```

```

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $name = trim($_POST["name"] ?? "");
    $email = trim($_POST["email"] ?? "");
    $age = trim($_POST["age"] ?? "");

    if ($name === "") {
        $nameError = "Name is required.";
    }

    if ($email === "") {
        $emailError = "Email is required.";
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $emailError = "Please enter a valid email.";
    }

    if ($age === "") {
        $ageError = "Age is required.";
    } elseif (!is_numeric($age)) {
        $ageError = "Age must be a number.";
    } elseif ($age < 18) {
        $ageError = "You must be at least 18.";
    }

    if ($nameError === "" && $emailError === "" && $ageError
=== "") {
        $successMessage = "Registration successful!";
    }
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Registration Validation</title>
</head>
<body>

```

```

<h2>Registration Form</h2>

<?php if ($successMessage !== ""): ?>
    <p style="color: green;">
        <?php echo htmlspecialchars($successMessage); ?>
    </p>
<?php endif; ?>

<form action="" method="post">
    <label>Name:</label>
    <input type="text" name="name" value="<?php echo htmlspecialchars($name); ?>">
    <span style="color: red;"><?php echo htmlspecialchars($nameError); ?></span>
    <br><br>

    <label>Email:</label>
    <input type="text" name="email" value="<?php echo htmlspecialchars($email); ?>">
    <span style="color: red;"><?php echo htmlspecialchars($emailError); ?></span>
    <br><br>

    <label>Age:</label>
    <input type="text" name="age" value="<?php echo htmlspecialchars($age); ?>">
    <span style="color: red;"><?php echo htmlspecialchars($ageError); ?></span>
    <br><br>

    <button type="submit">Register</button>
</form>

```

```
</body>
</html>
```

This example uses:

- `$_SERVER`
- `$_POST`
- `trim()`
- `filter_var()`
- `htmlspecialchars()`
- error messages
- form value preservation

▼ PHP Forms

1. What a PHP Form Does

A PHP form has two jobs:

Part	Job
HTML form	Takes input from user
PHP code	Receives and processes that input

Basic flow:

```
User enters values
↓
Clicks submit button
↓
PHP receives values using $_POST or $_GET
↓
PHP prints result
```

2. Basic Form Structure

A form is written using:

```
<form action="" method="post">

</form>
```

Example:

```
<form action="" method="post">
  <label>Enter Number:</label>
  <input type="number" name="number">

  <button type="submit">Submit</button>
</form>
```

Important parts

Part	Meaning
<code>form</code>	Starts the form
<code>action=""</code>	Submit to the same page
<code>method="post"</code>	Send data using POST
<code>input</code>	Takes user input
<code>name="number"</code>	PHP uses this name to receive data
<code>button type="submit"</code>	Sends the form

3. The Most Important Thing: **name**

This is extremely important.

```
<input type="number" name="ctl">
```

PHP receives it like this:

```
$ct1 = $_POST["ct1"];
```

The `name` must match.

Correct

```
<input type="number" name="marks">
```

```
$marks = $_POST["marks"];
```

Wrong

```
<input type="number" name="marks">
```

```
$marks = $_POST["student_marks"];
```

This is wrong because there is no input named `student_marks`.

4. `method="post"` and `$_POST`

Most exam-style PHP form questions use:

```
method="post"
```

So PHP receives values using:

```
$_POST["input_name"]
```

Example:

```

<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $name = $_POST["name"];
    echo $name;
}
?>

<form action="" method="post">
    <label>Name:</label>
    <input type="text" name="name">

    <button type="submit">Submit</button>
</form>

```

5. Why We Use

`$_SERVER["REQUEST_METHOD"]`

Before the user submits the form, `$_POST` has no data.

So we check:

```

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    // form was submitted
}

```

Full example:

```

<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    echo "Form submitted successfully.";
}
?>

<form action="" method="post">

```

```
<input type="text" name="username">
<button type="submit">Submit</button>
</form>
```

This prevents errors before the form is submitted.

6. Same-Page Form

This is the easiest and most useful format.

The form and PHP processing are in the **same file**.

```
<?php
$output = "";

if ( $_SERVER["REQUEST_METHOD"] === "POST" ) {
    $number = $_POST["number"];
    $output = "You entered: " . $number;
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Same Page Form</title>
</head>
<body>

<h2>Same Page PHP Form</h2>

<form action="" method="post">
    <label>Enter Number:</label>
    <input type="number" name="number">

    <br><br>
```



```

        <button type="submit">Submit</button>
    </form>

    <?php if ($output !== ""): ?>
        <h3><?php echo $output; ?></h3>
    <?php endif; ?>

</body>
</html>

```

7. Separate-Page Form

Sometimes the form is in one file and PHP processing is in another file.

File 1: **form.php**

```

<!DOCTYPE html>
<html>
<head>
    <title>Form Page</title>
</head>
<body>

    <h2>Input Form</h2>

    <form action="process.php" method="post">
        <label>Number:</label>
        <input type="number" name="number">

        <br><br>

        <button type="submit">Submit</button>
    </form>

```

```
</body>
</html>
```

File 2: `process.php`

```
<?php
$number = $_POST["number"];

echo "You entered: " . $number;
?>
```

Here:

```
action="process.php"
```

means the form sends data to `process.php`.

9. Receiving Number Inputs Correctly

Even if you use:

```
<input type="number">
```

PHP often receives the value as text.

So for calculations, convert it.

Integer

```
$marks = (int) $_POST["marks"];
```

Use `(int)` for whole numbers.

Float

```
$price = (float) $_POST["price"];
```

Use `(float)` for decimal numbers.

Example

```
<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $num1 = (int) $_POST["num1"];
    $num2 = (int) $_POST["num2"];

    echo "First Number: " . $num1 . "<br>";
    echo "Second Number: " . $num2 . "<br>";
}
?>

<form action="" method="post">
    <label>Number 1:</label>
    <input type="number" name="num1">

    <br><br>

    <label>Number 2:</label>
    <input type="number" name="num2">

    <br><br>

    <button type="submit">Submit</button>
</form>
```

10. Multiple Inputs in One Form

Example:

```
<form action="" method="post">
  <label>CT 1:</label>
  <input type="number" name="ct1">

  <br><br>

  <label>CT 2:</label>
  <input type="number" name="ct2">

  <br><br>

  <label>CT 3:</label>
  <input type="number" name="ct3">

  <br><br>

  <label>Midterm Marks:</label>
  <input type="number" name="midterm">

  <br><br>

  <label>Final Marks:</label>
  <input type="number" name="final">

  <br><br>

  <button type="submit">Calculate Total</button>
</form>
```

PHP receives them:

```
$ct1 = (int) $_POST["ct1"];
$ct2 = (int) $_POST["ct2"];
$ct3 = (int) $_POST["ct3"];
```

```
$midterm = (int) $_POST["midterm"];  
$final = (int) $_POST["final"];
```

11. Full Example: Form with Multiple Numeric Inputs

This example focuses on form handling only.

```
<?php  
$submitted = false;  
  
$value1 = 0;  
$value2 = 0;  
$value3 = 0;  
  
if ($_SERVER["REQUEST_METHOD"] === "POST") {  
    $submitted = true;  
  
    $value1 = (int) $_POST["value1"];  
    $value2 = (int) $_POST["value2"];  
    $value3 = (int) $_POST["value3"];  
}  
?>
```

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Multiple Input Form</title>  
</head>  
<body>  
  
    <h2>Multiple Number Input Form</h2>  
  
    <form action="" method="post">
```

```

<label>Value 1:</label>
<input type="number" name="value1" required>

<br><br>

<label>Value 2:</label>
<input type="number" name="value2" required>

<br><br>

<label>Value 3:</label>
<input type="number" name="value3" required>

<br><br>

<button type="submit">Submit</button>
</form>

<?php if ($submitted): ?>
    <hr>

    <h3>Submitted Values</h3>

    <p>Value 1: <?php echo $value1; ?></p>
    <p>Value 2: <?php echo $value2; ?></p>
    <p>Value 3: <?php echo $value3; ?></p>
<?php endif; ?>

</body>
</html>

```

12. Displaying Output After Submit

This part controls whether output appears.

```
<?php if ($submitted): ?>
    <h3>Output Here</h3>
<?php endif; ?>
```

Full idea:

```
<?php
$submitted = false;

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $submitted = true;
}
?>

<?php if ($submitted): ?>
    <p>The form was submitted.</p>
<?php endif; ?>
```

Before submit, output is hidden.

After submit, output is shown.

13. Showing Output in Paragraphs

Simple output:

```
<p>Total Marks: <?php echo $totalMarks; ?></p>
<p>Status: <?php echo $status; ?></p>
```

Example:

```
<?php if ($submitted): ?>
    <h3>Result</h3>

    <p>First Value: <?php echo $value1; ?></p>
```

```
<p>Second Value: <?php echo $value2; ?></p>
<p>Third Value: <?php echo $value3; ?></p>
<?php endif; ?>
```

16. Keeping Previous Input Values After Submit

Normally, after submitting, the form fields become empty again.

To keep values inside the form, use the `value` attribute.

```
<input type="number" name="marks" value="<?php echo $marks; ?>">
```

Full example:

```
<?php
$marks = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $marks = $_POST["marks"];
}
?>

<form action="" method="post">
    <label>Marks:</label>
    <input type="number" name="marks" value="<?php echo htmlspecialchars($marks); ?>">

    <button type="submit">Submit</button>
</form>
```

This is useful when you want the form to still show the submitted values.

17. Safer Way to Receive Inputs

Instead of directly writing:

```
$marks = $_POST["marks"];
```

You can write:

```
$marks = $_POST["marks"] ?? "";
```

Meaning:

| If `marks` exists, use it. Otherwise, use empty string.

Example:

```
$ct1 = $_POST["ct1"] ?? "";  
$ct2 = $_POST["ct2"] ?? "";  
$ct3 = $_POST["ct3"] ?? "";
```

For numbers:

```
$ct1 = (int) ($_POST["ct1"] ?? 0);  
$ct2 = (int) ($_POST["ct2"] ?? 0);  
$ct3 = (int) ($_POST["ct3"] ?? 0);
```

This prevents missing-input errors.

18. Safe Output with `htmlspecialchars()`

When printing text from user input, use:

```
htmlspecialchars()
```

Example:

```
echo htmlspecialchars($name);
```

This is especially important for text fields.

For numbers, this is usually fine:

```
echo $marks;
```

But for text, use:

```
echo htmlspecialchars($studentName);
```

19. Form Validation Basics

For simple exam questions, you may use HTML validation:

```
<input type="number" name="marks" required>
```

This means the field cannot be empty.

You can also use:

```
<input type="number" name="marks" min="0" max="100" required>
```

For decimal values:

```
<input type="number" name="price" step="0.01" required>
```

20. Button Text

The button text should match the question.

Examples:

```
<button type="submit">Calculate Total</button>
```

```
<button type="submit">Calculate</button>
```

```
<button type="submit">Submit</button>
```

```
<button type="submit">Check Result</button>
```

21. Using PHP Functions with Forms

Some questions say:

| Write a PHP function that takes three inputs.

The form still works the same way.

First, receive form data:

```
$a = (int) $_POST["a"];  
$b = (int) $_POST["b"];  
$c = (int) $_POST["c"];
```

Then pass them into a function:

```
$result = myFunction($a, $b, $c);
```

Example:

```
<?php  
function processValues($a, $b, $c) {  
    return $a + $b + $c;  
}
```

```

$submitted = false;
$result = 0;

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $submitted = true;

    $a = (int) $_POST["a"];
    $b = (int) $_POST["b"];
    $c = (int) $_POST["c"];

    $result = processValues($a, $b, $c);
}
?>

```

```

<!DOCTYPE html>
<html>
<head>
    <title>Function With Form</title>
</head>
<body>

<h2>Function With Form</h2>

<form action="" method="post">
    <label>A:</label>
    <input type="number" name="a" required>

    <br><br>

    <label>B:</label>
    <input type="number" name="b" required>

    <br><br>

    <label>C:</label>
    <input type="number" name="c" required>

```

```

        <br><br>

        <button type="submit">Submit</button>
    </form>

    <?php if ($submitted): ?>
        <h3>Result: <?php echo $result; ?></h3>
    <?php endif; ?>

</body>
</html>

```

The form part does not change.

Only the PHP calculation changes.

22. Best Basic Template for Your Questions

Use this as your main form structure.

```

<?php
$submitted = false;

// Default variables
$input1 = 0;
$input2 = 0;
$input3 = 0;

// Output variables
$result1 = 0;
$result2 = 0;

if ($_SERVER["REQUEST_METHOD"] === "POST") {

```

```

    $submitted = true;

    // Receive inputs
    $input1 = (int) ($_POST["input1"] ?? 0);
    $input2 = (int) ($_POST["input2"] ?? 0);
    $input3 = (int) ($_POST["input3"] ?? 0);

    // Do calculation here
    $result1 = $input1;
    $result2 = $input2;
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>PHP Form Template</title>
</head>
<body>

<h2>PHP Form Template</h2>

<form action="" method="post">
    <label>Input 1:</label>
    <input type="number" name="input1" value="<?php echo h
tmlspecialchars($input1); ?>" required>

    <br><br>

    <label>Input 2:</label>
    <input type="number" name="input2" value="<?php echo h
tmlspecialchars($input2); ?>" required>

    <br><br>

    <label>Input 3:</label>

```

```

<input type="number" name="input3" value="<?php echo h
tmlspecialchars($input3); ?>" required>

<br><br>

<button type="submit">Calculate</button>
</form>

<?php if ($submitted): ?>
    <hr>

    <h3>Output</h3>

    <table border="1" cellpadding="10">
        <tr>
            <th>Result 1</th>
            <th>Result 2</th>
        </tr>
        <tr>
            <td><?php echo $result1; ?></td>
            <td><?php echo $result2; ?></td>
        </tr>
    </table>
<?php endif; ?>

</body>
</html>

```

23. Form Knowledge Checklist

To answer your shown questions, you should understand these form topics:

Topic	Needed?
<form> tag	Yes

Topic	Needed?
<code>action=""</code>	Yes
<code>method="post"</code>	Yes
<code><input type="number"></code>	Yes
<code>name</code> attribute	Very important
Submit button	Yes
<code>\$_POST</code>	Very important
<code>\$_SERVER["REQUEST_METHOD"]</code>	Very important
Type casting with <code>(int)</code> and <code>(float)</code>	Very important
Showing output after submit	Yes
HTML table output	Yes
<code>required</code> attribute	Useful
<code>htmlspecialchars()</code>	Good habit
Same-page form	Very useful
Separate-page form	Sometimes useful

Key Idea

For every form-based PHP question, your form job is always:

```

Create input fields with name attributes
↓
Submit with method="post"
↓
Receive using $_POST["name"]
↓
Convert numbers using (int) or (float)
↓
Display output using echo

```